

Exercício – Implementar operações CRUD

Concluído

- 10 minutos

Vamos continuar estendendo nosso controlador de API Web para adicionar a capacidade de criar (`POST`), atualizar (`PUT`) e excluir (`DELETE`) pizzas de nosso estoque.

Adicionar uma pizza

Vamos permitir que uma pizza seja adicionada por meio da API da Web usando um método `POST` .

Substitua o comentário `// POST action` em `Controllers/PizzaController.cs` pelo seguinte código:

C#

```
[HttpPost]
public IActionResult Create(Pizza pizza)
{
    PizzaService.Add(pizza);
    return CreatedAtAction(nameof(Get), new { id = pizza.Id },
        pizza);
}
```

A ação anterior:

- Responde apenas ao verbo HTTP `POST` , conforme indicado pelo atributo `[HttpPost]` .
- Insere o objeto `Pizza` do corpo da solicitação no cache na memória.

Observação

Como o controlador está anotado com o atributo `[ApiController]` , está implícito que o parâmetro `Pizza` será encontrado no corpo da solicitação.

O primeiro parâmetro na chamada de método `CreatedAtAction` representa um nome de ação. A palavra-chave `nameof` é usada para evitar hard-coding do nome da ação. `CreatedAtAction` usa o nome da ação para gerar um cabeçalho de resposta

HTTP `location` com uma URL para a pizza recém-criada, conforme explicado na unidade anterior.

Modificar uma pizza

Agora, vamos permitir que uma pizza seja atualizada por meio da API da Web usando um método `PUT`.

Substitua o comentário `// PUT action` em *Controllers/PizzaController.cs* pelo seguinte código:

C#

```
[HttpPut("{id}")]
public IActionResult Update(int id, Pizza pizza)
{
    if (id != pizza.Id)
        return BadRequest();

    var existingPizza = PizzaService.Get(id);
    if(existingPizza is null)
        return NotFound();

    PizzaService.Update(pizza);

    return NoContent();
}
```

A ação anterior:

- Responde apenas ao verbo HTTP PUT, conforme indicado pelo atributo `[HttpPut]`.
- Requer que o valor do parâmetro `id` seja incluído no segmento da URL após `pizza/`.
- Retorna `IActionResult` porque o tipo de retorno `ActionResult` não é conhecido até o runtime. Os métodos `BadRequest`, `NotFound` e `NoContent` retornam os tipos `BadRequestResult`, `NotFoundResult` e `NoContentResult` respectivamente.

Observação

Como o controlador está anotado com o atributo `[ApiController]`, está implícito que o parâmetro `Pizza` será encontrado no corpo da solicitação.

Remover uma pizza

Por fim, vamos permitir que uma pizza seja removida por meio da API da Web usando um método `DELETE`.

Substitua o comentário `// DELETE action` em `Controllers/PizzaController.cs` pelo seguinte código:

C#

```
[HttpDelete("{id}")]
public IActionResult Delete(int id)
{
    var pizza = PizzaService.Get(id);

    if (pizza is null)
        return NotFound();

    PizzaService.Delete(id);

    return NoContent();
}
```

A ação anterior:

- Responde apenas ao verbo HTTP `DELETE`, conforme indicado pelo atributo `[HttpDelete]`.
- Requer que o valor do parâmetro `id` seja incluído no segmento da URL após `pizza/`.
- Retorna `IActionResult` porque o tipo de retorno `ActionResult` não é conhecido até o runtime. Os métodos `NotFound` e `NoContent` retornam os tipos `NotFoundResult` e `NoContentResult`, respectivamente.
- Consulta o cache na memória para uma pizza que corresponde ao parâmetro `id` fornecido.

Lembre-se de salvar o arquivo `Controllers/PizzaController.cs` antes de prosseguir,

Compilar e executar a API Web concluída

Compile e inicie a API Web executando o seguinte comando:

CLI do .NET

```
dotnet run
```

Testar a API Web concluída com arquivos HTTP

1. Reabra o arquivo **ContosoPizza.http** .
2. Faça uma solicitação **POST** para adicionar uma nova pizza a `HttpRepl` usando o seguinte comando:

Saída

```
POST {{ContosoPizza_HostAddress}}/pizza/  
Content-Type: application/json  
  
{  
  "name": "Hawaii",  
  "isGlutenFree": false  
}  
  
###
```

O comando anterior retorna a pizza recém-criada:

Saída

```
HTTP/1.1 201 Created  
Connection: close  
Content-Type: application/json; charset=utf-8  
Date: Wed, 17 Jan 2024 17:03:02 GMT  
Server: Kestrel  
Location: http://localhost:5192/Pizza/3  
Transfer-Encoding: chunked  
  
{  
  "id": 3,  
  "name": "Hawaii",
```

```
    "isGlutenFree": false
  }
```

3. Atualize a nova pizza `Hawaii` para uma pizza `Hawaiian` com uma solicitação `PUT` usando o seguinte comando:

Saída

```
PUT {{ContosoPizza_HostAddress}}/pizza/3
Content-Type: application/json

{
  "id": 3,
  "name": "Hawaiian",
  "isGlutenFree": false
}

###
```

O comando anterior retorna a seguinte saída que indica êxito:

Saída

```
HTTP/1.1 204 No Content
Connection: close
Date: Wed, 17 Jan 2024 17:07:30 GMT
Server: Kestrel
```

Para verificar se a pizza foi atualizada, execute novamente a ação `GET` usando o seguinte comando:

Saída

```
GET {{ContosoPizza_HostAddress}}/pizza/3
Accept: application/json

###
```

O comando anterior retorna a pizza recém-atualizada:

Saída

```
HTTP/1.1 200 OK
Connection: close
Content-Type: application/json; charset=utf-8
Date: Wed, 17 Jan 2024 17:09:01 GMT
Server: Kestrel
Transfer-Encoding: chunked

{
  "id": 3,
  "name": "Hawaiian",
  "isGlutenFree": false
}
```

4. Nossa API também pode excluir a pizza recém-criada por meio da ação `DELETE` se você executar o seguinte comando:

CLI do .NET

```
DELETE {{ContosoPizza_HostAddress}}/pizza/3

###
```

O comando anterior retorna um resultado `204 No Content` para êxito:

Saída

```
HTTP/1.1 204 No Content
Date: Fri, 02 Apr 2021 23:30:04 GMT
Server: Kestrel
```

Para verificar se a pizza foi removida, execute novamente a ação `GET` usando o seguinte comando:

CLI do .NET

```
GET {{ContosoPizza_HostAddress}}/pizza/  
Accept: application/json  
  
###
```

O comando anterior retorna as pizzas originais como resultados:

Saída

```
HTTP/1.1 200 OK  
Content-Type: application/json; charset=utf-8  
Date: Fri, 02 Apr 2021 23:31:15 GMT  
Server: Kestrel  
Transfer-Encoding: chunked  
  
[  
  {  
    "id": 1,  
    "name": "Classic Italian",  
    "isGlutenFree": false  
  },  
  {  
    "id": 2,  
    "name": "Veggie",  
    "isGlutenFree": true  
  }  
]
```

Você terminou de implementar e testar uma API Web recém-criada com ASP.NET Core.

Opcional: Testar a API Web concluída com o HTTPREPL da linha de comando

1. Reabra o terminal existente `httprepl` ou abra um novo terminal integrado do Visual Studio Code selecionando **Terminal>Novo Terminal** no menu principal.
2. Se você abriu um novo terminal, conecte-se à API Web executando o seguinte comando:

CLI do .NET

```
httprepl https://localhost:{PORT}
```

Outra opção é executar o seguinte comando a qualquer momento enquanto `HttpRepl` estiver em execução:

CLI do .NET

```
connect https://localhost:{PORT}
```

3. Vá até o ponto de extremidade `Pizza` executando o seguinte comando:

CLI do .NET

```
cd Pizza
```

4. Execute o seguinte comando para ver as novas ações na API de Pizza:

CLI do .NET

```
ls
```

O comando anterior mostra uma saída das APIs disponíveis para o ponto de extremidade `Pizza`:

Saída

```
https://localhost:{PORT}/Pizza> ls
.      [GET|POST]
..     []
{id}   [GET|PUT|DELETE]
```

5. Faça uma solicitação `POST` para adicionar uma nova pizza a `HttpRepl` usando o seguinte comando:

CLI do .NET

```
post -c '{"name":"Hawaii", "isGlutenFree":false}'
```

O comando anterior retorna a pizza recém-criada:

Saída

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Date: Fri, 02 Apr 2021 23:23:09 GMT
Location: https://localhost:{PORT}/Pizza?id=3
Server: Kestrel
Transfer-Encoding: chunked

{
  "id": 3,
  "name": "Hawaii",
  "isGlutenFree": false
}
```

6. Atualize a nova pizza `Hawaii` para uma pizza `Hawaiian` com uma solicitação `PUT` usando o seguinte comando:

CLI do .NET

```
put 3 -c '{"id": 3, "name": "Hawaiian",
"isGlutenFree": false}'
```

O comando anterior retorna a seguinte saída que indica êxito:

Saída

```
HTTP/1.1 204 No Content
Date: Fri, 02 Apr 2021 23:23:55 GMT
Server: Kestrel
```

Para verificar se a pizza foi atualizada, execute novamente a ação `GET` usando o seguinte comando:

CLI do .NET

```
get 3
```

O comando anterior retorna a pizza recém-atualizada:

Saída

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Date: Fri, 02 Apr 2021 23:27:37 GMT
Server: Kestrel
Transfer-Encoding: chunked

{
  "id": 3,
  "name": "Hawaiian",
  "isGlutenFree": false
}
```

7. Nossa API também pode excluir a pizza recém-criada por meio da ação `DELETE` se você executar o seguinte comando:

CLI do .NET

```
delete 3
```

O comando anterior retorna um resultado `204 No Content` para êxito:

Saída

```
HTTP/1.1 204 No Content
Date: Fri, 02 Apr 2021 23:30:04 GMT
Server: Kestrel
```

Para verificar se a pizza foi removida, execute novamente a ação `GET` usando o seguinte comando:

CLI do .NET

```
get
```

O comando anterior retorna as pizzas originais como resultados:

Saída

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Date: Fri, 02 Apr 2021 23:31:15 GMT
Server: Kestrel
Transfer-Encoding: chunked

[
  {
    "id": 1,
    "name": "Classic Italian",
    "isGlutenFree": false
  },
  {
    "id": 2,
    "name": "Veggie",
    "isGlutenFree": true
  }
]
```

Você terminou de implementar e testar uma API Web recém-criada com ASP.NET Core.